Université de Pau et des Pays de l'Adour - IUT DE BAYONNE - PAYS-BASQUE
Département Informatique

AUDOUARD Raphaël, GENTIL Rémi, LAGÜE Théo

année 2022-2023
SAE 2.01

TD2-TP4

SAÉ 2.01 – Développement d'une application

Lecteur de diaporamas – Dossier d'Analyse et conception

https://github.com/AirG33/LecteurDiaporama

1. Compléments de spécifications externes.

Nous n'avons pas eu de points flous durant la grande partie de la SAÉ. Or, lorsque des petits problèmes sont survenus nous sommes directement partis voir un professeur afin qu'il nous vienne en aide.

2. Scénarios

Description du scénario nominal et de un / deux scénarios alternatifs afin de mettre en évidence les interactions entre le système et l'utilisateur

Scénario nominal:

Utilisateur	Système	Maquette(s) associée(s)
L'utilisateur veut lancer le diaporama		LecteurVue
	le système affiche l'image du diaporama	
L'utilisateur passe à l'image suivante		
	Le système affiche l'image suivante du diaporama	
L'utilisateur passe à		

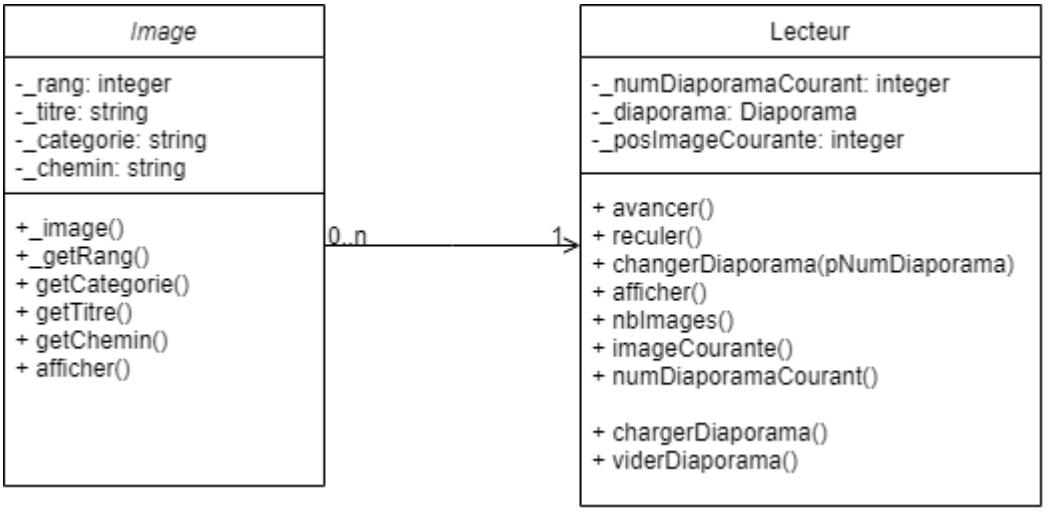
l'image précédente		
	le système affiche l'image précédente du diaporama	
L'utilisateur arrête le diaporama	le système efface l'image actuelle et stop le diaporama	

Scénario alternatif:

Utilisateur	Système	Maquette(s) associée(s)
L'utilisateur veut lancer le diaporama		
	Le système affiche l'image du diaporama	
L'utilisateur change le mode manuel en automatique		
	Le système passe du mode manuel en automatique	
	Le système fait défiler automatiquement les images à une vitesse défini	
L'utilisateur arrête le diaporama		
	Le système efface l'image actuelle et stop le diaporama	LecteurVue

3. Diagramme de classe (UML)

(a) Le diagramme de classes UML se focalise sur les classes **métier**, cad celles décrivant les éléments structurants de l'application, indépendamment des éléments d'interface.



(b) Dictionnaire des éléments pour chaque classe

Classe Image			
Nom attribut	**Signification**	**Type**	**Exemple**
`_rang`	Rang de l'image dans le diaporama actuel	`unsigned int`	`1`
`_titre`	Titre correspondant à l'image	`string`	`blanche neige`
`_categorie`	Categorie de l'image (type)	`string`	`animal`
`_chemin`	Chemin de l'image au répertoire défini qui l'a contient	`string`	`F://images/SAE2.01/`

Tableau 1 : Dictionnaire des éléments - Classe Image

Classe Lecteur			
Nom attribut	**Signification**	**Type**	**Exemple**
`_numDiaporamaCourant`	Numéro di diaporama actuel, défini par défaut à la valeur 0	`unsigned int`	`1`
`_diaporama`	pointeur vers les images du diaporama	`Diaporama`	`blanche neige`
`_posImageCourante`	Position de l'image actuelle dans le diaporama Commence a 0 et indéfini quand aucun diaporama chargé.	`unsigned int`	`1`

Tableau 2 : Dictionnaire des éléments - Classe Lecteur

```
#ifndef LECTEUR_H
#define LECTEUR_H
#include "image.h"
#include <vector>

typedef vector<Image*> Diaporama;   // Structure de données contenant les infos sur les images

class Lecteur
{
public:
    Lecteur();
    void avancer();             // incrémente _posImageCourante, modulo nbImages()
    void reculer();             // décrémente _posImageCourante, modulo nbImages()
    void changerDiaporama(unsigned int pNumDiaporama);    // permet de choisir un diaporama, 0 si aucun diaporama souhaité
    void afficher();            // affiche les informations sur lecteur-diaporama et image courante
    unsigned int nbImages();    // affiche la taille de _diaporama
    Image* imageCourante();     // retourne le pointeur vers l'image courante
    unsigned int numDiaporamaCourant();

private:
    unsigned _numDiaporamaCourant;   // numéro du diaporama courant, par défaut 0
    Diaporama _diaporama;            // pointeurs vers les images du diaporama
    unsigned int _posImageCourante;  /* position, dans le diaporama,
                                        de l'image courante.
                                        Indéfini quand diaporama vide.
                                        Démarre à 0 quand diaporama non vide */
private:
    void chargerDiaporama();    // charge dans _diaporama les images du _numDiaporamaCourant
    void viderDiaporama();      // vide _diaporama de tous ses objets image et les delete
};

#endif // LECTEUR_H
```

Figure 4 : Schéma de classe = Classe lecteur

```
#ifndef IMAGE_H
#define IMAGE_H
#include <iostream>
using namespace std;

class Image
{
public:
    Image(unsigned int pRang=0,
              string pCategorie="", string pTitre="", string pChemin = "");
    unsigned int getRang();
    string getCategorie();
    string getTitre();
    string getChemin();
    void afficher();           // affiche tous les champs de l'image

private:
    unsigned int _rang;         /* rang de l'image au sein du diaporama
                                   auquel l'image est associée */
    string _titre;              // intitulé de l'image
    string _categorie;          // catégorie de l'image (personne, animal, objet)
    string _chemin;             // chemin complet vers le dossier où se trouve l'image
};

#endif // IMAGE_H
```

Figure 5 : Schéma de classe = Classe image

Version v0 – Version console seule

4. Implémentation et tests

4.1 Implémentation

Liste et rôle des fichiers de cette version :

lecteur.h	Spécification de la classe Lecteur
lecteur.cpp	Corps de la classe Lecteur
image.h	Spécification de la classe Image
image.cpp	Corps de la classe Image
main.cpp	Test les méthodes de la classe Lecteur

4.2 Test

Voici le résultat qui été attendu:

```
Lecteur vide
Diaporama num. 1 selectionne.
4 images chargees dans le diaporama
Diaporama num. 1
image courante : image( rang:1, titre:Grincheux, categorie:personne, chemin:C:\cartesDisney\carteDisney1.gif)

Test avancer() : 4 fois
avancer() :
Diaporama num. 1
image courante : image( rang:2, titre:Cendrillon, categorie:personne, chemin:C:\cartesDisney\carteDisney4.gif)
avancer() :
Diaporama num. 1
image courante : image( rang:3, titre:Blanche Neige, categorie:personne, chemin:C:\cartesDisney\carteDisney2.gif)
avancer() :
Diaporama num. 1
image courante : image( rang:4, titre:Mickey, categorie:animal, chemin:C:\cartesDisney\carteDisney1.gif)
avancer() :
Diaporama num. 1
image courante : image( rang:1, titre:Grincheux, categorie:personne, chemin:C:\cartesDisney\carteDisney1.gif)

Test reculer() : 5 fois
reculer() :
Diaporama num. 1
image courante : image( rang:4, titre:Mickey, categorie:animal, chemin:C:\cartesDisney\carteDisney1.gif)
reculer() :
Diaporama num. 1
image courante : image( rang:3, titre:Blanche Neige, categorie:personne, chemin:C:\cartesDisney\carteDisney2.gif)
reculer() :
Diaporama num. 1
image courante : image( rang:2, titre:Cendrillon, categorie:personne, chemin:C:\cartesDisney\carteDisney4.gif)
reculer() :
Diaporama num. 1
image courante : image( rang:1, titre:Grincheux, categorie:personne, chemin:C:\cartesDisney\carteDisney1.gif)
reculer() :
Diaporama num. 1
image courante : image( rang:4, titre:Mickey, categorie:animal, chemin:C:\cartesDisney\carteDisney1.gif)

Enlever le diaporama courant = Choisir diaporama 0
0 images restantes dans le diaporama.
Lecteur vide
Press <RETURN> to close this window...
```

Ce que nous avons obtenu:

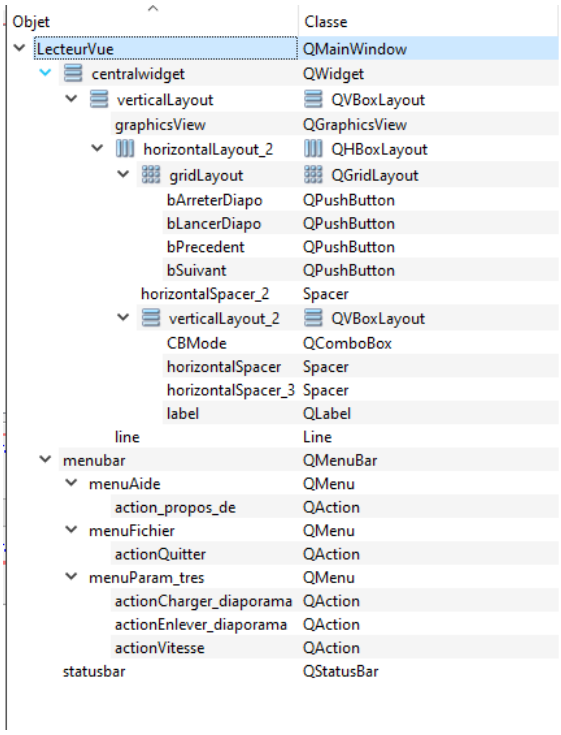
```
C:\Qt\Qt6.3.1\Tools\QtCreator\bin\qtcreator_process_stub.exe
Lecteur vide
Diaporama num. 1 selectionne.
4 images chargees dans le diaporama
Diaporama num.1
image courante : image( rang:1, titre:Grincheux, categorie:personne, chemin:C:\cartesDisney\carteDisney1.gif)

Test avancer() : 4 fois
avancer() :
Diaporama num.1
image courante : image( rang:2, titre:Cendrillon, categorie:personne, chemin:C:\cartesDisney\carteDisney4.gif)
avancer() :
Diaporama num.1
image courante : image( rang:3, titre:Blanche Neige, categorie:personne, chemin:C:\cartesDisney\carteDisney2.gif)
avancer() :
Diaporama num.1
image courante : image( rang:4, titre:Mickey, categorie:animal, chemin:C:\cartesDisney\carteDisney1.gif)
avancer() :
Diaporama num.1
image courante : image( rang:1, titre:Grincheux, categorie:personne, chemin:C:\cartesDisney\carteDisney1.gif)

Test reculer() : 5 fois
reculer() :
Diaporama num.1
image courante : image( rang:4, titre:Mickey, categorie:animal, chemin:C:\cartesDisney\carteDisney1.gif)
reculer() :
Diaporama num.1
image courante : image( rang:3, titre:Blanche Neige, categorie:personne, chemin:C:\cartesDisney\carteDisney2.gif)
reculer() :
Diaporama num.1
image courante : image( rang:2, titre:Cendrillon, categorie:personne, chemin:C:\cartesDisney\carteDisney4.gif)
reculer() :
Diaporama num.1
image courante : image( rang:1, titre:Grincheux, categorie:personne, chemin:C:\cartesDisney\carteDisney1.gif)
reculer() :
Diaporama num.1
image courante : image( rang:4, titre:Mickey, categorie:animal, chemin:C:\cartesDisney\carteDisney1.gif)

Enlever le diaporama courant = Choisir diaporama 0
0 images restantes dans le diaporama.
Lecteur vide
Press <RETURN> to close this window...
```

Testeur:	R.GENTIL	Date:	24/05/2023	
Elément Testé:	afficher()	Version:	1.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V1.0**
Valide N°1	affiche informations sur lecteur	numDiaporamaCourant > 0	image courante:	image courante:
Invalide N°1	Affiche informations sur le lecteur	numDiaporamaCourant <= 0	Echec	Diaporama vide

Testeur:	R.GENTIL	Date:	24/05/2023	
Elément Testé:	changerDiaporama()	Version:	1.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V1.0**
Valide N°1	affiche informations sur lecteur	numDiaporamaCourant > 0	image courante:	image courante:
Valide N°2	charger le diaporama courant	num DiaporamaCourant() > 0	diaporama chargé	diaporama chargé

Version v1 – projet Graphique seul

5. Éléments d'interface



Objet	Classe
LecteurVue	QMainWindow
centralwidget	QWidget
verticalLayout	QVBoxLayout
graphicsView	QGraphicsView
horizontalLayout_2	QHBoxLayout
gridLayout	QGridLayout
bArreterDiapo	QPushButton
bLancerDiapo	QPushButton
bPrecedent	QPushButton
bSuivant	QPushButton
horizontalSpacer_2	Spacer
verticalLayout_2	QVBoxLayout
CBMode	QComboBox
horizontalSpacer	Spacer
horizontalSpacer_3	Spacer
label	QLabel
line	Line
menubar	QMenuBar
menuAide	QMenu
action_propos_de	QAction
menuFichier	QMenu
actionQuitter	QAction
menuParam_tres	QMenu
actionCharger_diaporama	QAction
actionEnlever_diaporama	QAction
actionVitesse	QAction
statusbar	QStatusBar

6. Implémentation et tests

6.1 Implémentation

Liste et rôle des fichiers de cette version :

lecteurVue.h	Spécification de la classe graphique Qt contenant l'interface du lecteur de diaporamas
lecteurVue.cpp	Corps de la classe LecteurVue
lecteurvue.ui	Fichier du dessin de l'interface réalisé par QtDesigner
main.cpp	Teste les méthodes de la classe Lecteur

6.2 Test

Remarques sur l'implémentation :

Nous avons choisi de réaliser 4 slots qui sont : lancerDiapo, arreterDiapo, suivant et precedent.

Pour tous les slots nous avons choisi comme signal clicked car il faut un clic de l'utilisateur sur chaque bouton pour activer son action.

Testeur:	R.GENTIL	Date:	30/05/2023	
Elément Testé:	lancerDiapo()	Version:	1.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V1.0**
Valide N°1	qDebug test du bouton lancerDiapo	clique souris sur le bouton	le diapo se lance	le diapo se lance

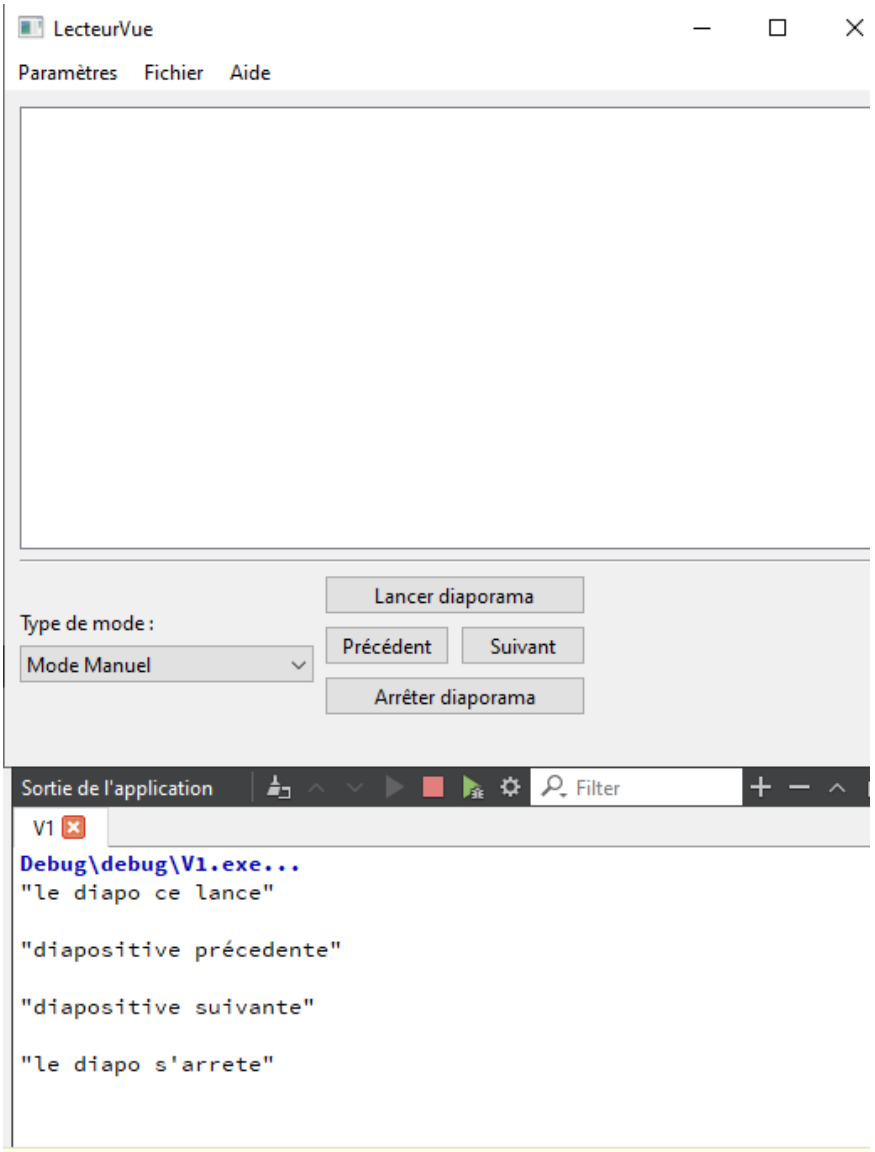
Testeur:	R.GENTIL	Date:	30/05/2023	
Elément Testé:	arreterDiapo()	Version:	1.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V1.0**
Valide N°1	qDebug test du bouton arreterDiapo	clique souris sur le bouton	le diapo s'arrete	le diapo s'arrete

Testeur:	R.GENTIL	Date:	30/05/2023	
Elément Testé:	suivant()	Version:	1.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V1.0**
Valide N°1	qDebug test du bouton suivant	clique souris sur le bouton	diapositive suivante	diapositive suivante

Testeur:	R.GENTIL	Date:	30/05/2023	
Elément Testé:	precedent()	Version:	1.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V1.0**
Valide N°1	qDebug test du bouton precedent	clique souris sur le bouton	diapositive precedent	diapositive precedent

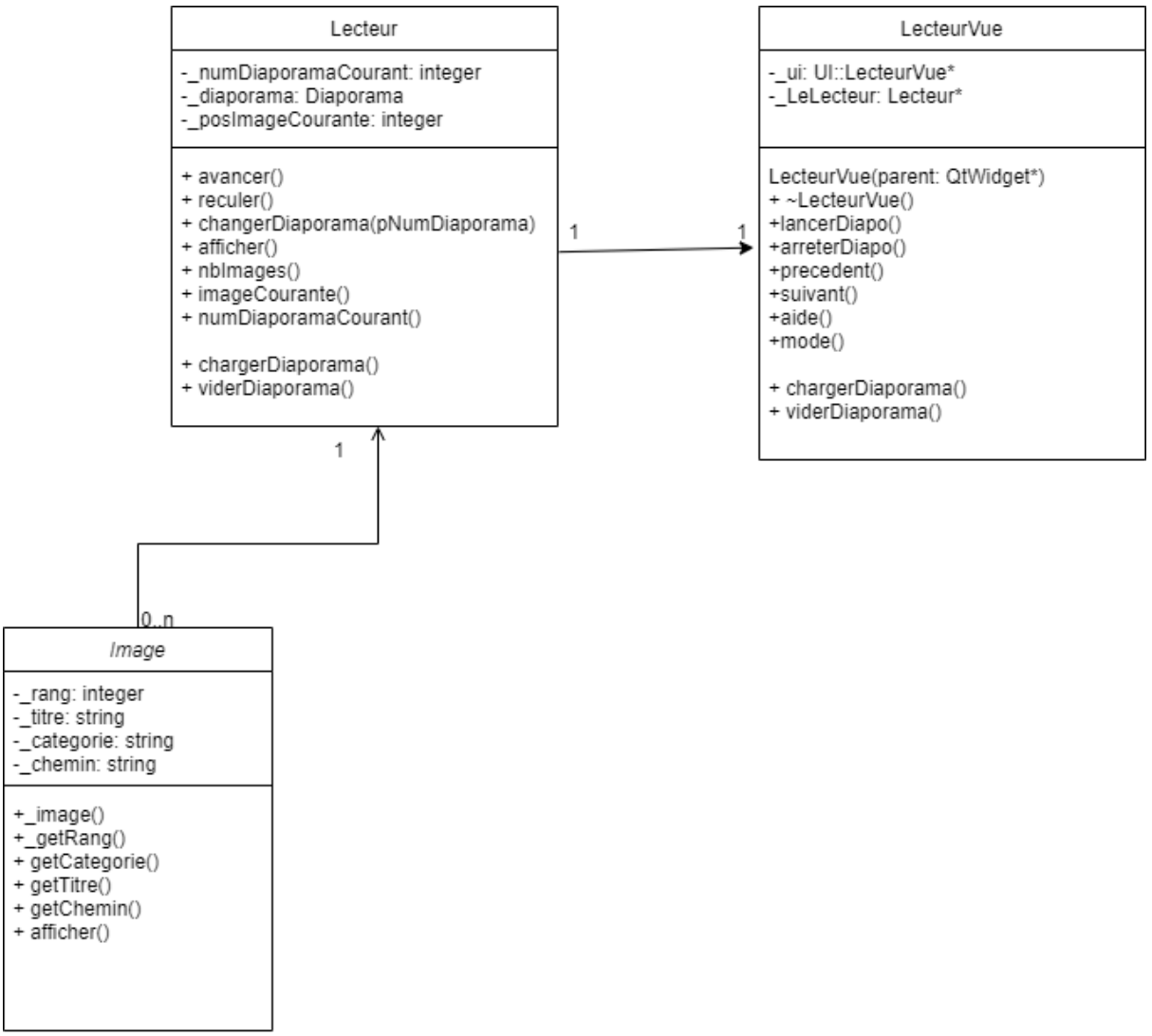
Les test effectués se font depuis les intéractions avec les boutons: Lancer diaporama, Précedent, Suivant et Arreter Diaporama.
Ils sont testés grâce aux retours écrit dans le terminal grâce à des qDebug, qui vérifient que l'action de chaque bouton fonctionne.

Ce que nous avons obtenu:



Version v2 –

7. Diagramme de classes (UML)



8. Comportement de l'application

8.1 Diagramme états-transitions-actions du lecteur de diaporamas (v2)

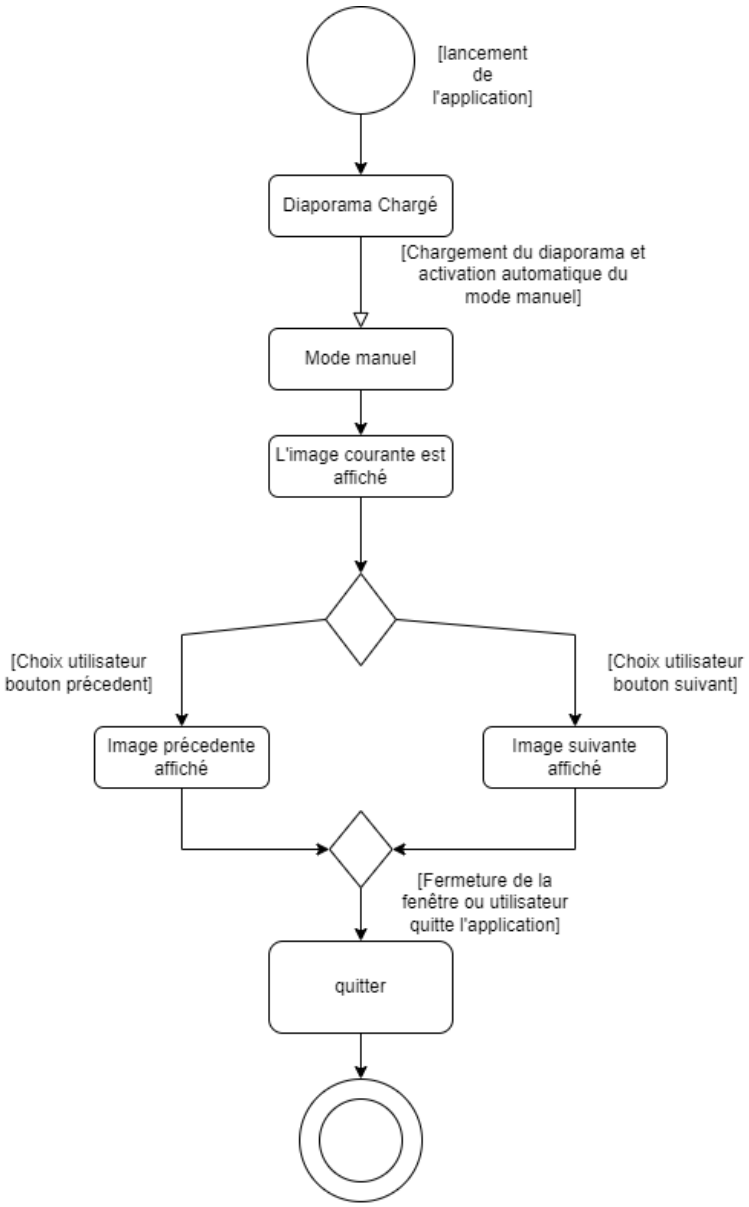


Figure 9 : Diagramme états-transitions du lecteur de diaporamas – v2

8.2 Dictionnaire des états, événements et Actions (v2)

Dictionnaire des états du diaporama

nomEtat	*Signification*
`Image courante affiché`	cet état permet d'afficher l'image qui est actuellement dans le diaporama
`quitter`	cet état permet de fermer l'application

Tableau 2 : États du lecteur de diaporamas – v2

Dictionnaire des événements faisant changer le diaporama d'état

nomEvénement	*Signification*
`lancement application`	cet état est activé lorsque l'utilisateur lance l'application
`chargement diaporama`	cet état est activé lorsque l'utilisateur charge un diaporama depuis l'application
`clique bouton suivant`	cet état est activé lorsque l'utilisateur clique sur le bouton "Suivant" pour pouvoir afficher l'image suivante dans le diaporama
`clique bouton précédent`	cet état est activé lorsque l'utilisateur clique sur le bouton "Précédent" pour pouvoir afficher l'image précédente dans le diaporama
`Fermeture fenetre`	Cet état est activé lorsque l'utilisateur quitte l'application par le bouton prévue à cet effet qui cause la fermeture de l'application

Tableau 3 : Événements faisant changer le diaporama d'état – v2

Description des actions réalisées lors de la traversée des transitions:

Pour ce diagramme états-évenements-actions: nous n'avons pas de transition entre les différentes actions

8.3 Table T_EtatsEvenementsActions (v2)

Élément graphique prenant en charge cet événement à	bouton Suivant	bouton Precedent	quitter	charger diaporama
Événement à *nomEtat*	suivant	precedent	fermeture fenetre	chargement diaporama
image courante affiché	Affiche l'image suivante	Affiche l'image précédente		Affiche la première image du diaporama
quitter			fermeture de l'application	

Tableau 5 : Matrice d'états-transitions du lecteur de diaporamas – v2

9. Implémentation et tests

9.1 Implémentation (v2)

Liste et rôle des fichiers de cette version :

lecteurVue.h	Spécification de la classe graphique Qt contenant l'interface du lecteur de diaporamas Définit la structure et le comportement de la fenêtre principale de l'application de lecteur de diaporamas
lecteurVue.cpp	Corps de la classe LecteurVue.
lecteurvue.ui	Fichier du dessin de l'interface réalisé par QtDesigner
lecteur.h	Spécification de la classe Lecteur. Définit les fonctions et les membres nécessaires pour la gestion du diaporama
lecteur.cpp	Corps de la classe Lecteur
image.h	Spécification de la classe Image Définit les propriétés et les opérations liées à une image individuelle du diaporama
image.cpp	Corps de la classe Image
main.cpp	Lancement de l'application et de l'affichage de son interface utilisateur.

- `lecteurVue.h` et `lecteurVue.cpp` définissent la classe `LecteurVue`, qui représente l'interface utilisateur du lecteur de diaporamas. Ils spécifient la structure et le comportement de la fenêtre principale de l'application.

- `lecteurvue.ui` est un fichier généré par Qt Designer qui représente le dessin de l'interface utilisateur. Il définit la disposition des widgets et leurs propriétés visuelles.

- `lecteur.h` et `lecteur.cpp` définissent la classe `Lecteur`, responsable de la gestion du diaporama. Ils incluent des fonctions pour charger les images, afficher les diapositives et gérer la lecture automatique ou manuelle du diaporama.

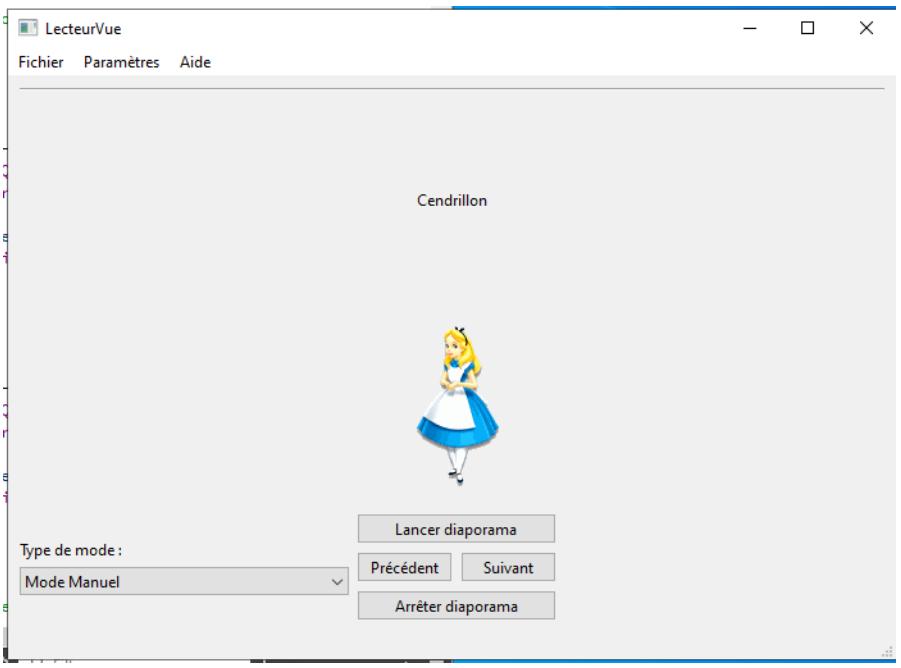
- `image.h` et `image.cpp` définissent la classe `Image`, représentant une image individuelle du diaporama. Ils spécifient les propriétés et les opérations liées à une image, telles que le chemin d'accès, les métadonnées et les manipulations d'image.

- `main.cpp` est le point d'entrée de l'application. Il initialise l'application Qt, crée une instance de la classe `LecteurVue`, configure les signaux et les slots, et lance la boucle d'événements pour démarrer l'application.

9.2 Tests (v2)

Testeur:	R.GENTIL	Date:	24/05/2023	
Elément Testé:	suivant()	Version:	2.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V2.0**
Valide N°1	affiche l'image suivante dans le diaporama	clique sur suivant	Affichage image suivante	le diaporama affiche l'image suivante
Invalide N°1	affiche l'image suivante dans le diaporama	imageCourante <= 0	Echec	Diaporama vide

Testeur:	R.GENTIL	Date:	24/05/2023	
Elément Testé:	precedent()	Version:	2.0	
Classe	**Description**	**Valeurs rentrée**	**Résultats attendus**	**Résultats obtenus V2.0**
Valide N°1	affiche l'image précedente dans le diaporama	clique sur precedent	Affichage image precedente	le diaporama affiche l'image precedente
Invalide N°1	affiche l'image precedente dans le diaporama	imageCourante <= 0	Echec	Diaporama vide



Voici les tests et résultats obtenus.
Nous en déduisons donc que cette version est fonctionnelle et correspond aux attentes.
Nous avons fait des tests par rapport aux principales commandes qui sont “Suivant” et “Précédent”.
Elles sont valides lorsqu’un diaporama est chargé, et il est impossible d’appuyer sur les boutons sans diaporama chargés.

Les boutons “Lancer diaporama” et “Arreter diaporama” ont des messages qui s’inscrivent dans la console lorsqu’ils sont appuyés car ils ne servent que lors de l’activation du mode automatique qui est mis en place à partir de la V3. Mais ces boutons sont tout de même fonctionnels.

Version v3 –

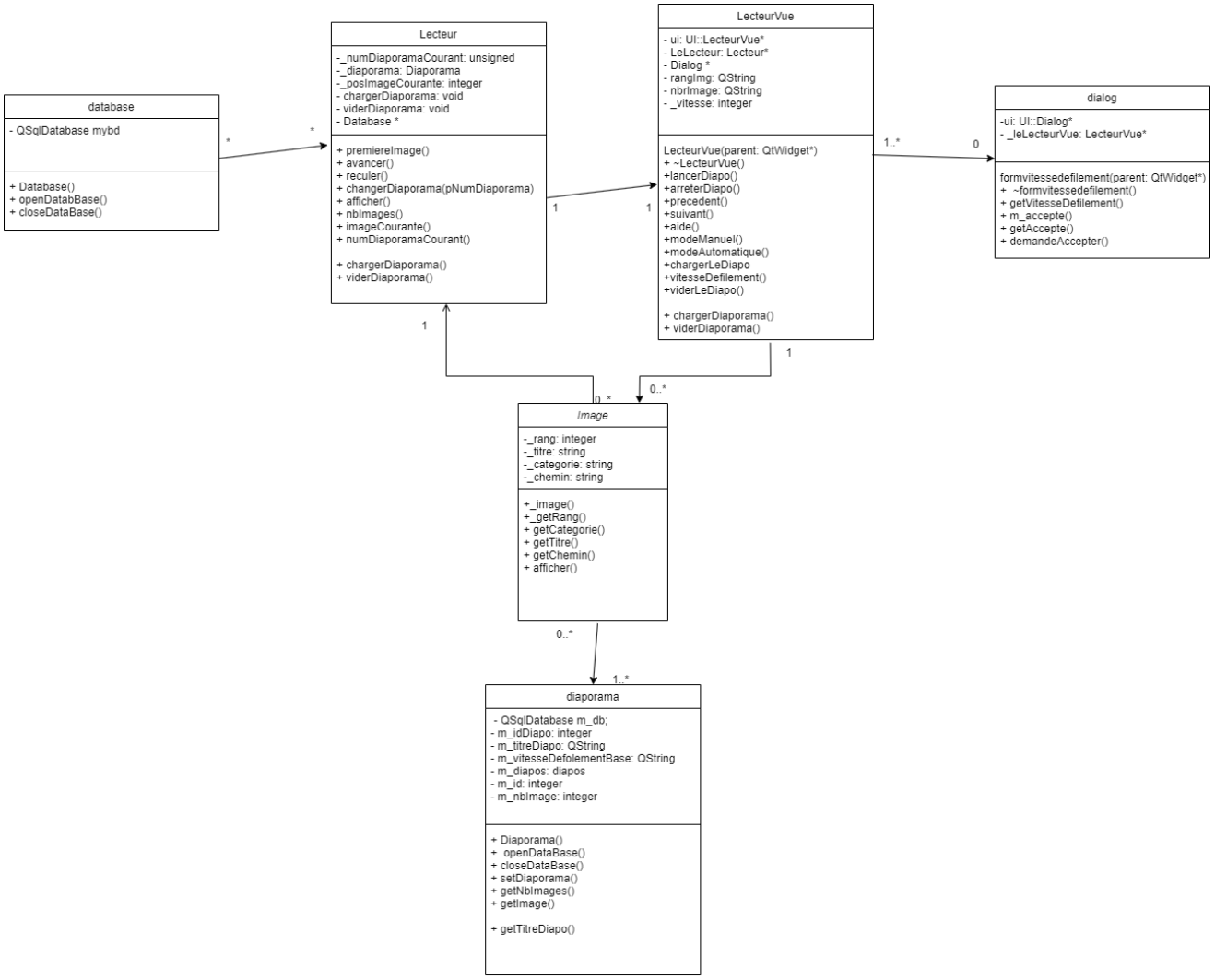
Pour cette version, l'objectif était de mettre en place le mode automatique et que celui-ci soit fonctionnel.
Pour le mode automatique nous avons initialisé le bouton lancerDiapo à false et le bouton arreterDiapo à true car lorsque le mode est lancé on ne peut que s'arrêter.
Pour le mode Manuel nous avons initialisé le bouton lancerDiapo à true et le bouton arreterDiapo à false et les boutons precedent et suivant aussi à true car si on lance le diapo le mode automatique s'active et le bouton arreterDiaporama est disponible mais si l'on active les boutons précedent ou suivant, le mode automatique s'enlève.

Version v4 –

Pour cette version, il fallait mettre en place une option pour changer la vitesse de défilement. Dans la V3, en mode automatique, le diaporama passait toujours à l'image suivante toutes les 2 secondes. Désormais, dans la V4, l'option actionVitesse ouvre une fenêtre de type Dialog qui permet de régler la vitesse avec la spinbox SBVitesseDefil. Celle-ci prend une valeur de 1 à 6 (par défaut à 1, ce qui correspond au délai de 2 secondes utilisé auparavant) et plus cette valeur est grande, plus le délai est court.

Version v5 –

10. Diagramme de classes (UML)



11. Comportement de l'application

11.1 Diagramme états-transitions-actions du lecteur de diaporamas (v5)

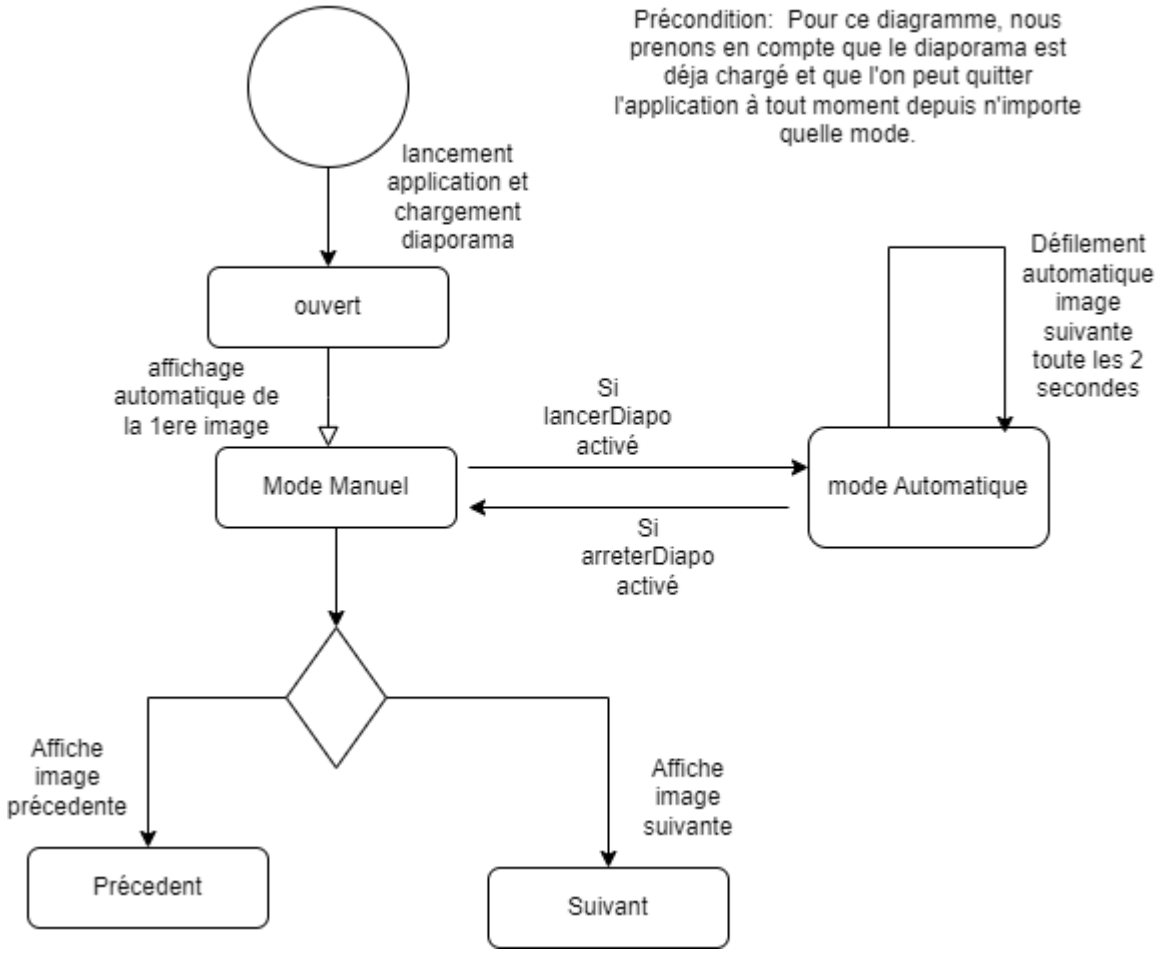


Figure 9 : Diagramme états-transitions du lecteur de diaporamas – v5

11.2 Dictionnaire des états, événements et Actions (v5)

Dictionnaire des états du diaporama

nomEtat	*Signification*
`ouvert`	Le diaporama est ouvert
`Mode Manuel`	Le lecteur à un diaporama a lire et l'utilisateur agit pour passer les images
`mode Automatique`	Le lecteur à un diaporama a lire et les images défilent automatiquement
`Suivant`	L'utilisateur à l'option de passer à l'image suivante dans le diaporama
`Précédent`	L'utilisateur à l'option de passer à l'image précédente dans le diaporama

Tableau 2 : États du lecteur de diaporamas – v5

Dictionnaire des événements faisant changer le diaporama d'état

nomEvénement	*Signification*
`lancement application et chargement diaporama`	permet de lancer le lecteur et d'y charger un diaporama
`affichage automatique de la première image`	affiche la première image dans le lecteur
`Affichage image précédente`	passe à l'image précédente en mode manuel
`Affichage image suivante`	passe à l'image suivante en mode manuel
`Si lancerDiapo activé`	met le diaporama en mode automatique
`si arreterDiapo activé`	enlève le mode automatique et bascule en mode manuel
`Défilement automatique image suivante toute les 2 secondes`	Permet de faire défiler les images automatiquement en mode Automatique

Tableau 3 : Événements faisant changer le diaporama d'état – v5

Description des actions réalisées lors de la traversée des transitions:

Pour ce diagramme états-évenements-actions: nous n'avons pas de transition entre les différentes actions

8.3 Table T_EtatsEvenementsActions (v5)

Élément graphique prenant en charge cet événement à	bouton Précedent	bouton Suivant	bouton lancerDiapo	bouton arreterDiapo
nomEtat	bouton Précedent	bouton Suivant	bouton lancerDiapo	bouton arreterDiapo
ouvert	affiche la première image	affiche la première image	affiche la première image	affiche la première image
Mode Manuel	Affiche l'image précédente	Affiche l'image suivante	x	x
mode Automatique	x	x	Lance le diaporama en automatique	Arrete le diaporama en automatique
Précedent	Action précedent en mode Manuel	x	x	x
Suivant	x	Action suivant en mode Manuel	x	x

Tableau 5 : Matrice d'états-transitions du lecteur de diaporamas – v5

12. Implémentation et tests

12.1 Implémentation (v5)

Liste et rôle des fichiers de cette version :

lecteurVue.h	Spécification de la classe graphique Qt contenant l'interface du lecteur de diaporamas Définit la structure et le comportement de la fenêtre principale de l'application de lecteur de diaporamas

lecteurVue.cpp	Corps de la classe LecteurVue
lecteurvue.ui	Fichier du dessin de l’interface réalisé par QtDesigner
lecteur.h	Spécification de la classe Lecteur Définit les fonctions et les membres nécessaires pour la gestion du diaporama
lecteur.cpp	Corps de la classe Lecteur
image.h	Spécification de la classe Image Définit les propriétés et les opérations liées à une image individuelle du diaporama
image.cpp	Corps de la classe Image
main.cpp	Lancement de l'application et de l'affichage de son interface utilisateur. Il initialise l'application Qt, crée une instance de la classe "LecteurVue"
Dialog.h	Permet de configurer dialog qui est une fenêtre pour la vitesse de défilement
Dialog.cpp	Corps de la classe dialog
Dialog.ui	Fichier du dessin de l’interface réalisé par QtDesigner pour la vitesse de défilement

Remarques sur l'implémentation :

`lecteurVue.h` : Spécifie la classe graphique Qt contenant l'interface du lecteur de diaporamas. Définit la structure et le comportement de la fenêtre principale de l'application de lecteur de diaporamas.

`lecteurVue.cpp` : Contient le code source (corps) de la classe `LecteurVue`, qui implémente les fonctionnalités et les comportements spécifiés dans `lecteurVue.h`.

`lecteurvue.ui` : Fichier généré par Qt Designer représentant le dessin de l'interface utilisateur du lecteur de diaporamas. Il définit la disposition des widgets, leur apparence et leurs propriétés.

`lecteur.h` : Spécifie la classe `Lecteur`, qui définit les fonctions et les membres nécessaires pour la gestion du diaporama

`main.cpp` : Point d'entrée de l'application, il initialise l'application Qt, crée une instance de la classe `LecteurVue`, configure les signaux et les slots, et lance la boucle d'événements pour démarrer l'application.

`Dialog.cpp` : Contient le code source (corps) de la classe `Dialog`, qui implémente les fonctionnalités et les comportements spécifiés dans `Dialog.h`.

12.2 Tests (v5)

1ème : Ce code configure une base de données en spécifiant le type de connexion, le nom de la base de données, le nom d'utilisateur et le mot de passe associés à cette base de données.

```
mydb = QSqlDatabase::addDatabase(CONNECT_TYPE);
mydb.setDatabaseName(DATABASE_NAME);
mydb.setUserName("tlague_bd");
mydb.setPassword("tlague_bd");
```

2ème : En résumé, ce code met à jour les éléments d'interface utilisateur avec des informations sur le nombre total d'images et le rang de l'image courante. Il permet finalement de faire un compteur pour le nombre d'image.

```
ui->nbr_total_image->setText(nbrImage);
rangImg.setNum(LeLecteur->imageCourante()->getRang());
ui->nbr_image->setText(rangImg);
```

3ème : Ce code initialise une connexion à une base de données, prépare une requête SQL pour récupérer certaines colonnes de deux tables, et stocke la requête dans l'objet "query" pour une utilisation ultérieure.

```
Image* imageACharger;
mydb = new Database;
mydb->openDataBase();
QSqlQuery query;
QString MaRequete("SELECT idphoto, nomFamille, titrePhoto, uriPhoto "
                  "FROM Diapos "
                  "JOIN Familles ON Diapos.idFam = Familles.idFamille ");
```

4ème : Ce code exécute une requête SQL nommée "MaRequete" et récupère les résultats à l'aide d'un objet "query". Si la requête est exécutée avec succès, le code parcourt les résultats de la requête à l'aide d'une boucle "for"

```
if (query.exec(MaRequete))
{
    for(int i = 0; query.next(); i++)
    {
        imageACharger = new Image(query.value(0).toInt(),
                                  query.value(1).toString().toStdString(),
                                  query.value(2).toString().toStdString(),
                                  ":/v2" + query.value(3).toString().toStdString());
        _diaporama.push_back(imageACharger);
    }
}
```

5ème : Il concerne le comportement liée à l'interface des aspects fonctionnels du programme. Il permet de mettre en place un tri à bulles. Il compare les éléments adjacents dans le tableau en fonction de leur rang, et les échange si nécessaire pour les trier dans l'ordre croissant. Après l'exécution du code, le tableau sera trié en fonction du rang des images.

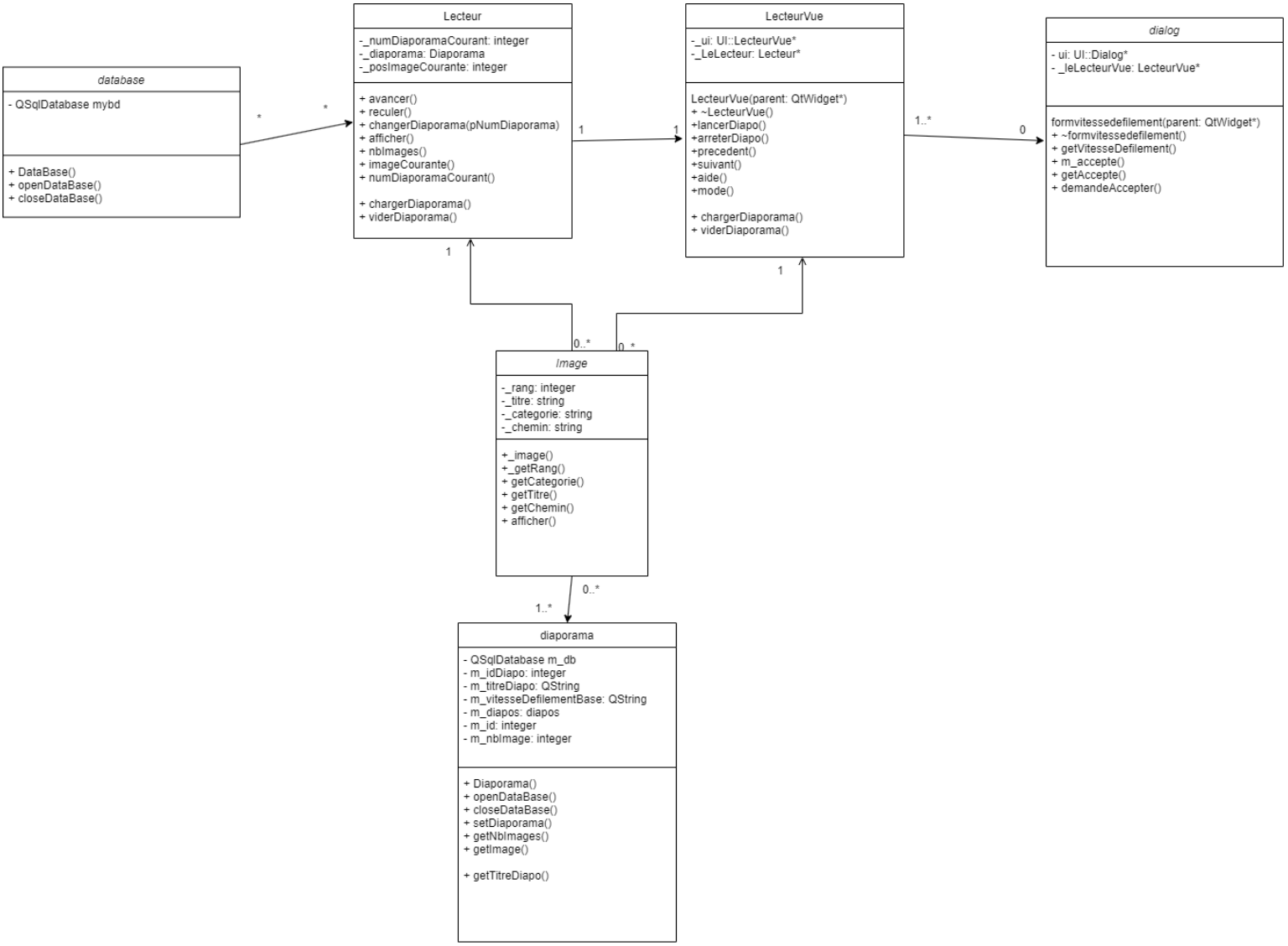
```
    Image* copieImage;

    for(unsigned int i = 0; i < nbImages() - 1; i++)
    {
        for(unsigned int j = 0; j < nbImages() - 1; j++)
        {
            if (_diaporama[j]->getRang() > _diaporama[j+1]->getRang())
            {
                copieImage = _diaporama[j];
                _diaporama[j] = _diaporama[j+1];
                _diaporama[j+1] = copieImage;
            }
        }
    }

     _posImageCourante=0;

     cout << "Diaporama num. " << numDiaporamaCourant() << " selectionne. " << endl;
     cout << nbImages() << " images chargees dans le diaporama" << endl;

}
```

7ème test : Il concerne le comportement non liée à l'interface des aspects fonctionnels du programme. Ce test permet avec le DNS de se connecter à la base de données.

```
#define DATABASE_NAME "tlague_bd_diapo"
#define CONNECT_TYPE "QODBC"
```

Version v6 –

13. Diagramme de classes (UML)



14. Comportement de l’application

14.1 Diagramme états-transitions-actions du lecteur de diaporamas (v6)

Le diagramme états-transitions-actions du lecteur de diaporamas est identique à celui de la V5 au stade de notre avancement de la V6 (voir 15.3 Commentaires)..

15. Implémentation et tests

15.1 Implémentation (v6)

Liste et rôle des fichiers de cette version :

lecteurVue.h	Spécification de la classe graphique Qt contenant l’interface du lecteur de diaporamas Définit la structure et le comportement de la fenêtre principale de l'application de lecteur de diaporamas
lecteurVue.cpp	Corps de la classe LecteurVue
lecteurvue.ui	Fichier du dessin de l’interface réalisé par QtDesigner
lecteur.h	Spécification de la classe Lecteur Définit les fonctions et les membres nécessaires pour la gestion du diaporama

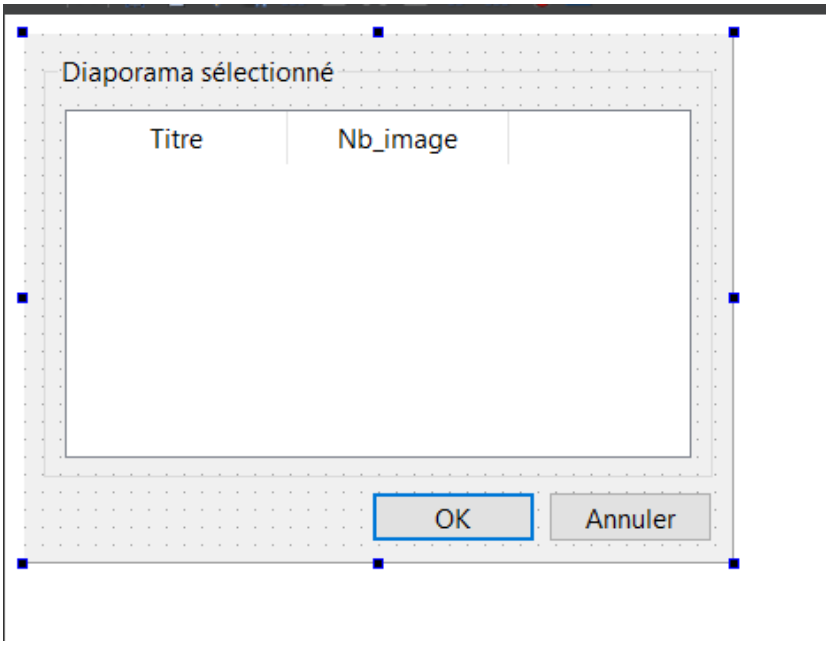
lecteur.cpp	Corps de la classe Lecteur
image.h	Spécification de la classe Image Définit les propriétés et les opérations liées à une image individuelle du diaporama
image.cpp	Corps de la classe Image
main.cpp	Lancement de l'application et de l'affichage de son interface utilisateur. Il initialise l'application Qt, crée une instance de la classe "LecteurVue"
Dialog.h	Permet de configurer dialog qui est une fenêtre pour la vitesse de défilement
Dialog.cpp	Corps de la classe dialog
diaporama.h	Définit la classe diaporama avec ses membres et ses méthodes.
diaporama.cpp	Corps de la classe diaporama
Dialog.ui	Fichier du dessin de l’interface réalisé par QtDesigner pour la vitesse de défilement
Diaporama.ui	Fichier du dessin de l’interface réalisé par QtDesigner pour la partie qui consiste à choisir quel diaporama on veut charger

Remarques sur l’implémentation :

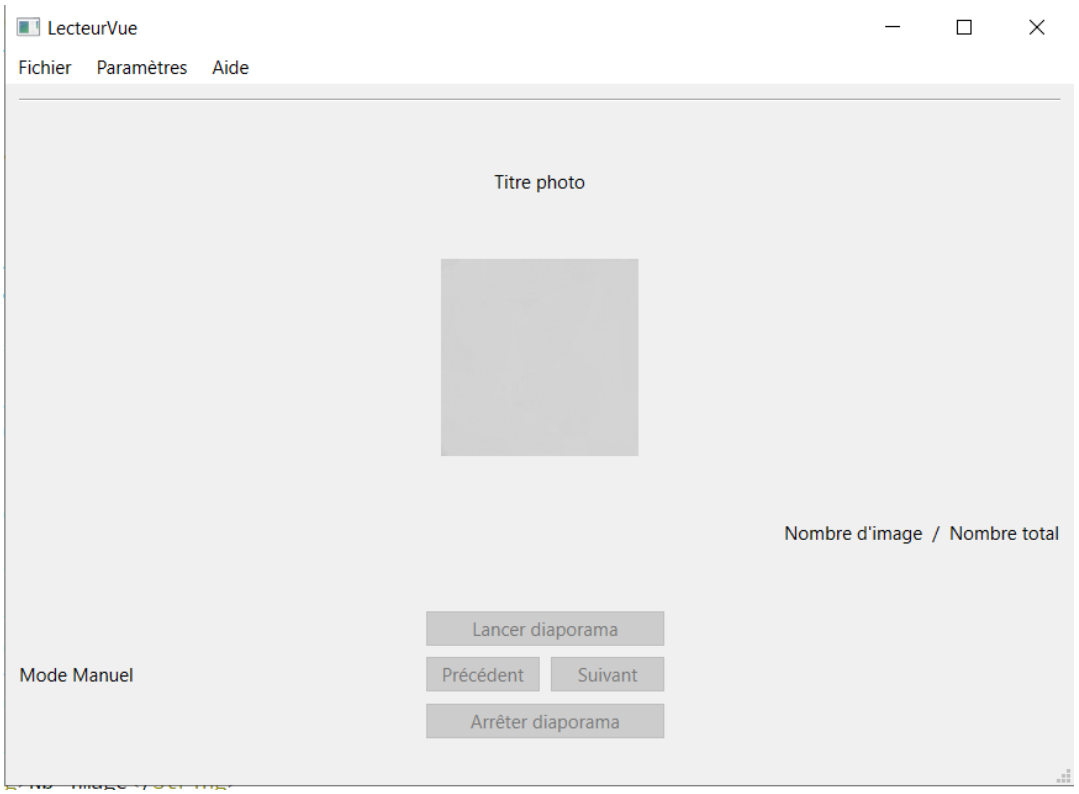
Le fichier "diaporama.h" définit la classe "Diaporama" avec ses membres et ses méthodes, tandis que le fichier "diaporama.cpp" implémente ces méthodes en leur donnant une fonctionnalité concrète. Ensemble, ces deux fichiers fournissent la structure et le comportement nécessaires pour choisir quel diaporama choisir et donc en fonction du choix afficher les bonnes images dedans.

15.2 Tests (v6)

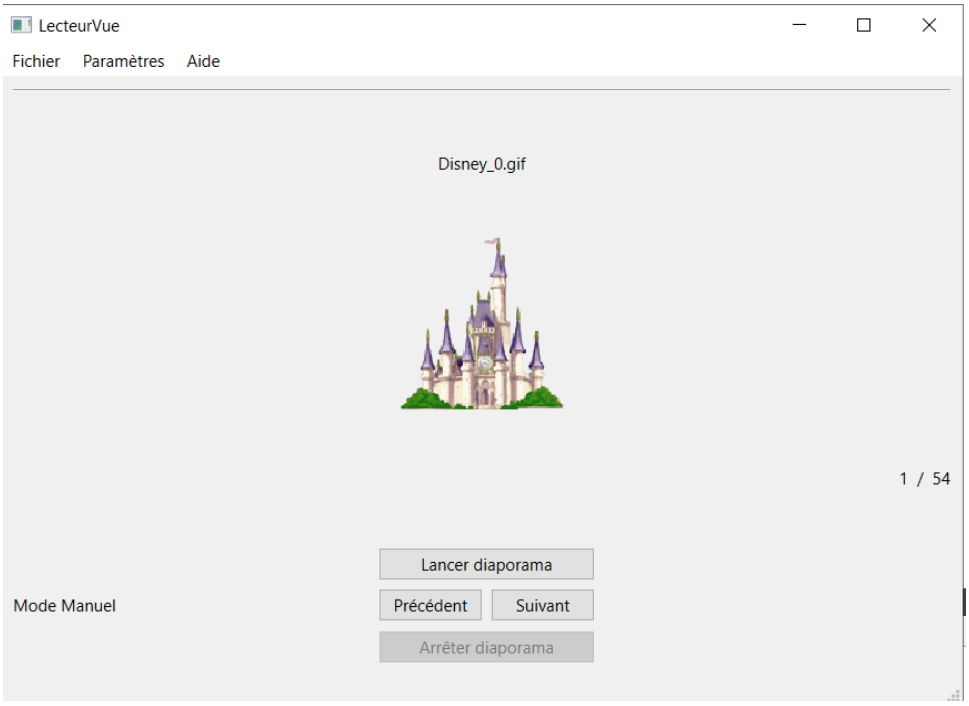
Interface graphique du choix de diaporama :



Interface graphique du lecteurvue au lancement du programme :



Interface graphique après l'action de charger diaporama :



15.3 Commentaires

Tout d'abord, nous créerons une nouvelle classe appelée "Diaporama" qui représentera un diaporama dans notre application. Cette classe sera responsable de la gestion des informations relatives au diaporama et aux images qui y sont associées.

Elle contiendra l'ID du diaporama, son titre et sa vitesse de défilement en attributs privés.

Nous allons modifier l'implémentation existante de la fonction "Charger diaporama" dans le menu "Fichier" de notre application. Cette fonction permettra de charger les informations relatives au diaporama et aux images à partir de la base de données.

Lorsque l'utilisateur choisit cette option dans le menu, une fenêtre de sélection s'affiche, lui permettant de choisir le diaporama qu'il souhaite charger.

Les informations du diaporama sélectionné, telles que le nom du diaporama et les images qui y sont associées, seront récupérées depuis la base de données.

Du côté de la base de données, nous devrons mettre en place les fonctionnalités nécessaires pour stocker les informations relatives aux différents diaporamas.

Les informations de chaque diaporama, telles que son nom et les images associées, devront être enregistrées dans des tables ou des documents spécifiques dans la base de données.

Lorsque l'utilisateur charge un diaporama, il devra être en mesure de sélectionner celui qu'il souhaite parmi les diaporamas disponibles dans la base de données, tels que "diaporama Pantxika", "diaporama Thierry", "diaporama Yann" et "diaporama Manu".

Lors du développement de cette partie de la base de données, nous avons été confrontés à certaines difficultés majeures. Malheureusement, malgré nos efforts, nous n'avons pas réussi à mettre en place la requête permettant de choisir le diapo souhaité et d'obtenir les photos associées en fonction de ce choix. En dépit de nos tentatives pour résoudre ces problèmes, nous n'avons pas pu surmonter ces difficultés spécifiques. Cela peut être dû à un manque de connaissances approfondies en matière de requêtes complexes ou à des

contraintes de temps et de ressources qui ne nous ont pas permis d'explorer toutes les options possibles.

16. Bilan

Dépôt Git où trouver le projet complet:

https://github.com/AirG33/LecteurDiaporama.git

Nous avons fait pour le rendu du 05/06/2023 toutes les versions jusqu'à la V5.

environ 30 commits ont été effectués sur ce repository

Temps global de travail:

Nous avons travaillé lors des séances de SAE ce qui représente 24 heures de projet au sein de l'iut, puis depuis chez nous, nous avons travaillé environ 30 heures en groupe ou à distance hors IUT.

Ce qui ferait un total de 54 heures de travail pour ce projet.

Apprentissages majeurs:

Lors de cette SAE nous avons pu apprendre la gestion entre de nombreuses interfaces graphiques, la gestion des bases de données au sein de code ainsi que l'inclusion d'images et leurs modifications dans une interface graphique. Le fait d'importer les images depuis une base de données vers Qt était tout nouveau pour nous mais en plus de ça nous avons aussi appris à manipuler ces données qui en l'occurrence étaient des images.

Difficultés majeures:

Nos principales difficultés ont été rencontrées notamment lors dès premières versions (V0 à V2) ou nous avons eu beaucoup de mal à démarrer et comprendre réellement l'objectif et les méthodes à mettre en place. Nous avons aussi rencontré beaucoup de difficultés au niveau de la réflexion qui au sein du groupe était plutôt lente car nous avions eu du mal à trouver les solutions pour parvenir aux objectifs ce qui a donc nécessité de nombreuses modifications et améliorations lors de travaux réalisés hors horaires prévus au sein de l'IUT. Durant la V2 nous avons remarqué qu'on avait un gros retard sur la documentation attendu pour cette SAÉ. Nous avons donc décidé de nous séparer en 2 "groupe" le premier était constitué de Raphael AUDOUARD ainsi que Théo LAGÜE et nous nous occupions du code principalement. Le 2ème groupe était donc composé essentiellement de Rémi Gentil qui s'occupé de la doc tout en suivant l'avancé du code.

Points positifs / négatifs de l’activité:

Les points positifs ont été le choix du thème qui est intéressant et donne envie de réaliser le projet,

Pour ce qui est des points négatifs nous pourrions citer le manque d’accompagnement par certains enseignants lors de séances encadrés ainsi que le formalisme de certaines demandes qui n’était pas forcément clair pour nous.

Il y a eu aussi pour notre part un réel manque de temps pour pouvoir atteindre tous les objectifs de manière rigoureuses. Ce qui a donc fait que nous ne sommes pas arrivé à la fn des versions demandés

Tableau de la répartition horaire de la SAE entre les membres:

	Théo Lagüe	Raphaël Audouard	Rémi Gentil
Documentation	2h	1h	9h
V0	7h	4h	5h
V1	4h	1h	3h
V2	6h	3h	4h
V3	2h	4h	1h
V4	2h	4h	1h
V5	6h	3h	1h
V6	5h	4h	1h